GET vs POST

In

Report Generation Systems

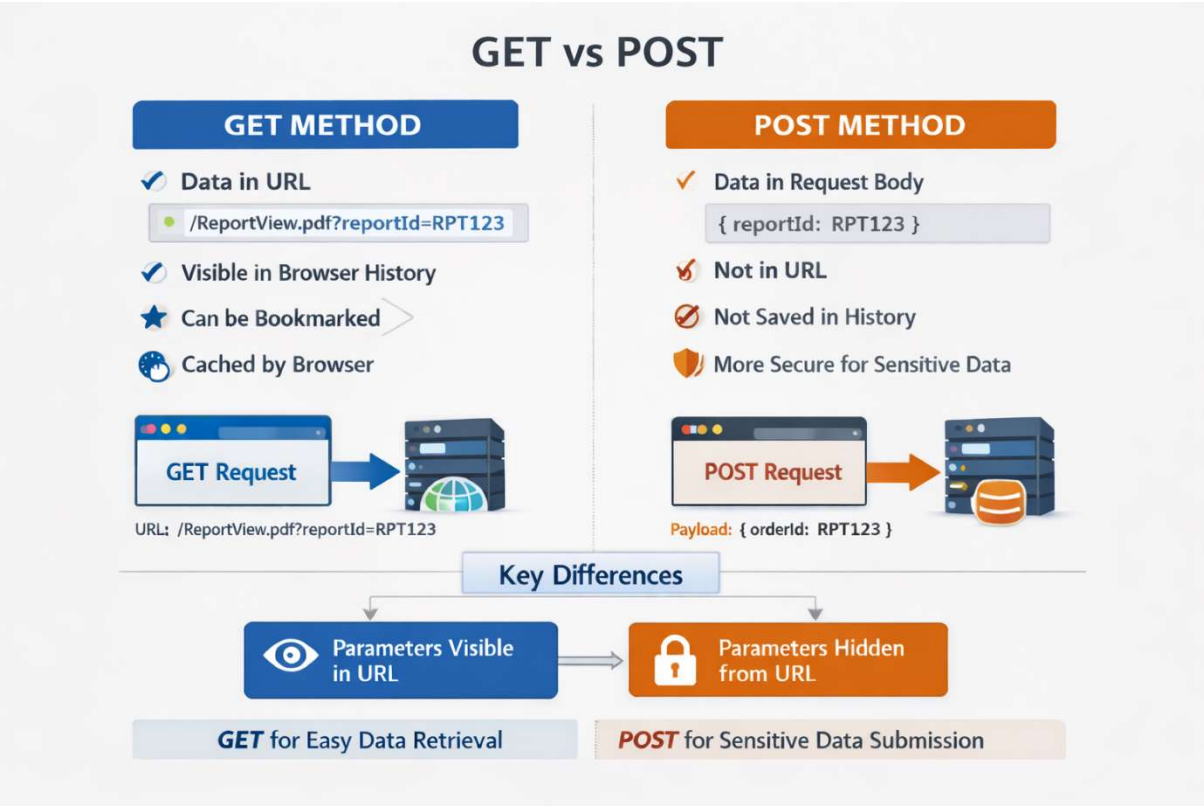
By

Muthu Kumar A

Understanding GET vs POST

In web applications, GET and POST are HTTP methods used to send data between the client (browser) and the server. The selection of these methods plays a critical role in report generation, system behavior, and security exposure.

It is important to understand that **the HTTP method (GET or POST) does NOT replace the need for proper server-side authentication and authorization validation**.



What Happens When the Page is Refreshed?

Refresh After POST Request.

When a report is generated using **POST**, the payload is sent in the **request body** (not in the URL).

If the user refreshes the page:

- Most browsers display: **"Confirm Form Resubmission"**

- If confirmed → Browser re-sends the POST request with the same body
- If not → It may send a GET request instead (depending on how the page was reached).

This may result in:

- Duplicate report generation
- Re-execution of heavy backend logic
- Duplicate transactions (if not properly controlled)

Refresh After GET Request.

When a page is requested using the GET method, the data (payload) is sent through the URL as query parameters.

Example:

/ReportView.pdf?reportId=RPT123 Here, reportId=RPT123 are the payload.

If the user refreshes the page:

- The same GET request is sent again
- No confirmation message appears
- The report reloads normally

Since GET is idempotent, refresh behavior is predictable.

What Happens If User Copies URL After POST?

Let us assume a report is generate using POST with the following URL:

POST /SalarySlipEmp.pdf (payload: empId=986301XXXXX).

After report loads, in browser URL may show as "/SalarySlipEmp.pdf "

When the copied URL is pasted into the browser, it becomes a GET request without any payload. Since the user is copied the URL of POST method (Payload is not visible in the URL).

In POST-based report generation, copying the browser URL does not replicate the original request because the payload is not embedded in

the address. However, the final behavior depends on how the server manages session state and report identifiers.

Report will:

- Either fail,
- Or show error,
- Or redirect to login,
- Or show default page,
- Execute and give the full details without any validation.

Note: POST data is:

- Sent in request body.
- Not appended to URL.
- Not copied when URL is shared.
- Not stored in bookmarks.

So copying URL does NOT reproduce the original report request.

What Happens If User Copies URL After GET?

In report generation systems, when **GET** is used, parameters are appended directly to the URL: "**/ReportView.pdf?reportId=RPT123"**
This means:

- The parameter is visible in the address bar
- It is stored in browser history
- It can be bookmarked
- It can be copied and shared

If another user opens the same URL, the browser sends the exact same request to the server and receive the same report.

Now the real question is not about GET. It is about **authorization**.
If the server validates:

- Who the user is (authentication)

- Whether the user is allowed to access that specific report (authorization)

Then the system remains secure, otherwise it introduces a security risk.

However, if the application checks login status but fails to verify ownership of the requested object, this can lead to **Broken Access Control**, commonly known as an IDOR scenario.

- The URL is reproducible.
- The request is replayable.
- The risk depends entirely on server-side validation.

Security Considerations.

In many application security discussions, a common question arises:

"Is POST more secure than GET?"

Especially in report generation systems where sensitive documents like salary slips, invoices, or financial reports are involved.

The answer is **No**.

The security of report generation systems does not depend on whether GET or POST is used; it depends entirely on proper server-side validation and access control. Simply hiding parameters in a POST request does not make an application secure.

The backend must always enforce strong authentication, object-level authorization, and ownership verification for every request.

If identifiers are predictable or ownership checks are missing, vulnerabilities such as Insecure Direct Object Reference (IDOR), parameter tampering, and enumeration attacks may occur.

Additionally, sensitive reports must be protected with strict cache-control policies to prevent unintended storage in browsers, proxies, or shared systems. Ultimately, security is enforced at the server layer, not by the choice of HTTP method.

Conclusion

- The decision between GET and POST is an architectural choice.
- It is not a security control.
- True security in report generation systems depends on:
  - Strong backend validation.
  - Proper authorization checks.
  - Ownership verification.
  - Secure caching policies.
- Never rely on the HTTP method to protect sensitive data.
- Security must always be enforced at the server.

*****End of the Document *****